Drifting Model

ADASP Reading Group

Sicheng Mao

2026/2/16

Outlines

1. Generative modeling as transform
2. Generate step by step
3. Issues
4. Rethinking temporality
5. The Drifting model
6. Training objective
7. Designing Drift field
8. What’ s the cost?

Generative modeling as transform

Generative modeling: sample from a simple distribution and transform it into the data distribution

Sample view: for each sample z , learn how to transform it to data x with $f : z \to x$

Distribution view: for simple distribution $q(z)$, learn how to transform it to data distribution with the pushforward mapping $f_{\#} : q(z) \to p_{data}(x)$

Generate step by step

Divide and conquer: model the transform f little by little

- VAEs, GANs: no divide and conquer.
- Normalizing Flows: f as a composition chain of invertible mappings:

$$f = f_N \circ \cdots \circ f_1$$

  deeper the network, better the expressivity.

- Diffusion/Flow Matching models: f as a composition chain of neural mappings with explicit time indices:

$$f = f(t, \cdot) \circ \cdots \circ f(0, \cdot)$$

  more the timesteps, less the noise.

Issues

Single-step generation

- VAEs: Blurry
- GANs: Hard to train
- NFs: not so promising expressivity due to invertibility and the costly Jacobian.

Multi-step generation

- Diffusion models: too slow. Accelerate with better Neural Diffrential Equation solver.
- Flow matching models: accelerate by ODE path instead of SDE path.

Can we have a good generation model that is natively single-step?

Rethinking temporality

- Temporality based on structure: NFs, one layer one composition
- Temporality based on time index: Diffusion models, one timestep one composition

But there is a temporality of the training itself: what if we have a model that learns how to do transformation gradually with the training steps?

The Diffusion paradigm: learn the difference between two timesteps, where the timesteps are explicitly sampled from a SDE/ODE trajectory.

$$z = x_0 \xrightarrow{f(0)} x_{t_1} \xrightarrow{f(t_1)} x_{t_2} \rightarrow \cdots \rightarrow x_{t_{N-1}} \xrightarrow{f(t_{N-1})} x_1$$

An alternative paradigm: learn how to gradually transform, where the timesteps are implicit, they are just the training steps!

$$z = x_0 \xrightarrow{f(0)} x_{t_1}, x_0 \xrightarrow{f(t_1)} x_{t_2}, \cdots, x_0 \xrightarrow{f(t_{N-1})} x_1$$

The Drifting model

- At each training step i, we know the origin point x_0, we know how to tranform it to $x_i = f(x_0)$, we want to know how to transform it to $x_{i+1} = x_i + \Delta x$, i.e. the **drift** Δx between two training steps, for every position x_i, that transforms the current source distribution q_i at timestep i **approaching** the data distribution p.

That is to say, knowing the drift field $\Delta x := \mathbf{V}_{p,q_i}(x_i)$. (Now we actually don't move the particles, we play with fields.)

- An necessary condition for such field $q = p \implies \mathbf{V}_{p,q}(x) = 0$
- If you achieve it, you no more drift it.
- We hope the sufficient direction holds too. $\mathbf{V}_{p,q}(x) = 0 \implies q = p$, so that the zero-drifting field is a criterion for convergence. It **approximately** does, with a good selection of the field. (See Appendix)

Training objective

- Intuitively, we should train the model to zero the field, i.e. the loss function

$$\mathbb{E}_{\epsilon}\left[\|\mathbf{V}_{p,q_{\theta_i}}\left(f_{\theta_i}(\epsilon)\right)\|^2\right]$$

- But backpropgation through a field of distributions is non-trivial, hence a fixed-point relation trick

$$f_{\hat{\theta}}(\boldsymbol{\epsilon}) = f_{\hat{\theta}}(\boldsymbol{\epsilon}) + \mathbf{V}_{p,q_{\hat{\theta}}}\left(f_{\hat{\theta}}(\boldsymbol{\epsilon})\right).$$

turns the loss to

$$\mathbb{E}_{\epsilon}\Big[\big\| \underbrace{f_\theta(\epsilon)}_{\text{prodiction}} - \underbrace{\texttt{stopgrad}\big(f_\theta(\epsilon) + \mathbf{V}_{p,q_\theta}\left(f_\theta(\epsilon)\right)\big)}_{\text{frozen target}}\big\|^2\Big]$$

which avoids BP through the distribution but only through the function.

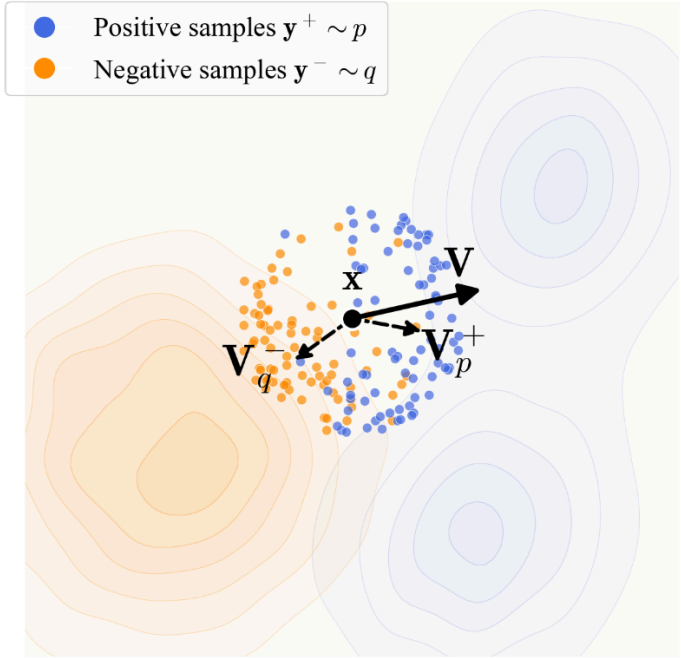
Designing Drift field

- Intuition: data distribution pulling and source distribution pushing (Just like electric field with positive charges and negative charges)

$$
\begin{aligned}
\mathbf{V}_{p,q}(\mathbf{x}) &:= \mathbf{V}_p^+(\mathbf{x}) - \mathbf{V}_q^-(\mathbf{x}) \\
\mathbf{V}_p^+(\mathbf{x}) &:= \frac{1}{Z_p}\mathbb{E}_p\left[k(\mathbf{x}, \mathbf{y}^+)(\mathbf{y}^+ - \mathbf{x})\right] \\
\mathbf{V}_q^-(\mathbf{x}) &:= \frac{1}{Z_q}\mathbb{E}_q\left[k(\mathbf{x}, \mathbf{y}^-)(\mathbf{y}^- - \mathbf{x})\right]
\end{aligned}
$$

- With typical Gaussian kernel:

$$
k(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{1}{\tau}\|\mathbf{x} - \mathbf{y}\|\right)
$$



What’s the cost?

The final form of the field

$$\mathbf{V}_{p,q}(\mathbf{x}) = \frac{1}{Z_p Z_q} \mathbb{E}_{p,q}\left[k(\mathbf{x}, \mathbf{y}^+)k(\mathbf{x}, \mathbf{y}^-)(\mathbf{y}^+ - \mathbf{y}^-)\right]$$

To better approximate the field, a big batch of data points is needed for low variance Monte Carlo, e.g. 4096 or 8192.

Misc

- drifting in VAE/RAE latent space
- drifting with feature-space loss

Thank you